Welcome back! Ab tak humne models ko instruct karna (Prompts) aur unke output ko clean format mein lana (Output Parsers) seekh liya hai.
Jab hum HealthBuddy jaise multi-module healthcare ecosystems banate hain, jahan predictive analytics aur Neo4j jaise graph databases use hote hain, toh AI ke liye patient ki chat history session ke across yaad rakhna crucial hota hai. Ek raw LLM yeh automatically nahi kar sakta. Yahin par **LangChain Memory** picture mein aati hai.
Aaj hum memory concepts aur message history ko deep dive karenge, using the latest LangChain LCEL architecture.

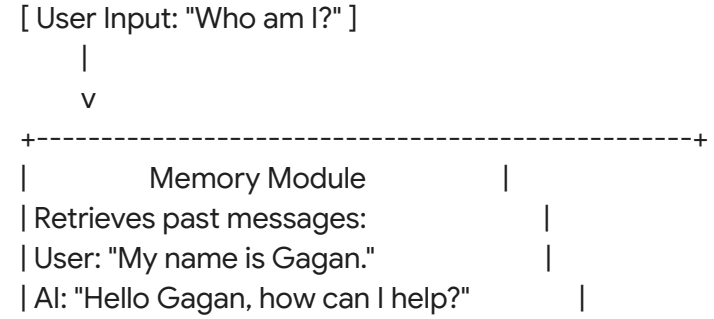
Module 1: The Core Concept of Memory (Topics 1, 2, 3)

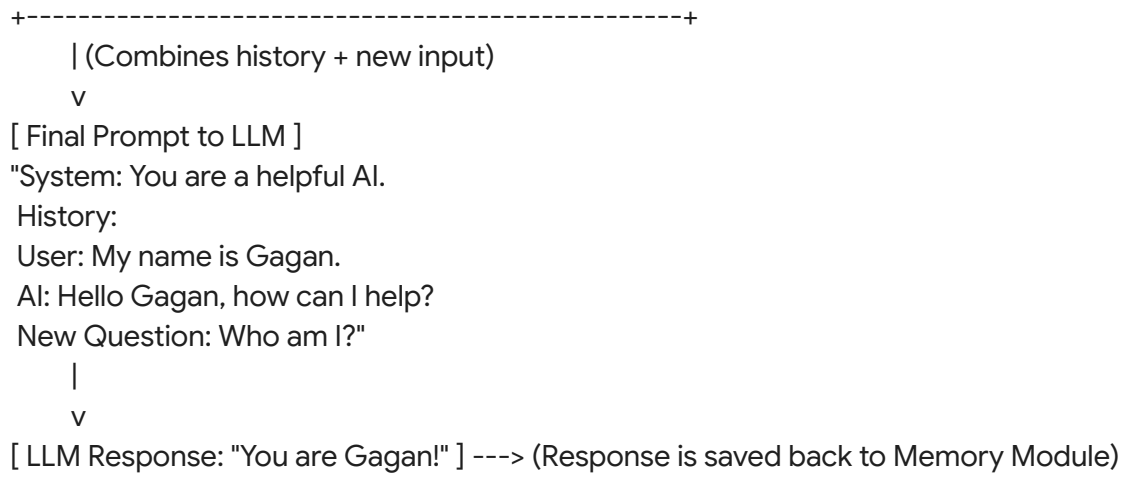
1. Why LLMs forget previous conversations?

- **Real-world Analogy:** Agar aap backend mein Spring Boot ya modern microservices architecture par kaam karte hain, toh aapko pata hoga ki REST APIs by design **stateless** hoti hain. Har HTTP request fresh hoti hai.
  LLMs bilkul waise hi hote hain. Yeh pure mathematical functions hain: f(prompt) = response. Jab aap Model ko prompt bhejte ho, toh API connection close hote hi model sab bhool jata hai. Uske paas apna koi "brain storage" ya internal RAM nahi hota jahan wo purani baatein save kare.

2. What is Memory in LangChain? & 3. How it stores chat history

Kyunki LLM stateless hai, humein usko har nayi request ke sath purani **poori conversation history** bhejni padti hai.
LangChain mein Memory ek system hai jo user aur AI ke beech ke messages ko ek List/Array mein store karta hai. Jab user naya question poochta hai, toh LangChain background mein pichli baaton ko ek lambi string mein convert karke naye question ke aage chipka deta hai (append kar deta hai) aur LLM ko bhejta hai.
Internal Architecture / Data Flow Diagram:





Module 2: Types of Memory & Limitations (Topics 4, 5, 6, 7, 8)

4. Different types of memory

LangChain alag-alag scenarios ke liye multiple memory types provide karta hai:

1. **ConversationBufferMemory:** Saare messages as-it-is save karta hai.
2. **ConversationBufferWindowMemory:** Sirf last k messages yaad rakhta hai (e.g., last 5 interactions).
3. **ConversationSummaryMemory:** Pichli baaton ko background mein ek LLM se summarize karwa leta hai taaki token size chota rahe.

5. ConversationBufferMemory in depth & 6. Message history concepts

Yeh sabse basic aur common memory hai. Yeh data ko HumanMessage aur AIMessage objects ki ek list mein rakhta hai.
Modern Note: Purane LangChain mein hum specifically ConversationBufferMemory class use karte the. Latest LCEL (LangChain Expression Language) mein, is concept ko implement karne ke liye hum **InMemoryChatMessageHistory** ka use karte hain. Yeh exact same "buffer" logic follow karta hai par architecture mein fast aur modular hai.

7. Context window limitations (The biggest trap!)

Har LLM ki ek token limit hoti hai (e.g., GPT-3.5 ki 4K ya 16K, Gemini ki 1M+). Agar aap ek lamba chat session chalate ho (e.g., teaching DSA concepts for hours) aur standard Buffer memory use karte ho, toh history list itni badi ho jayegi ki:

1. LLM API token limit exceed kar degi aur crash ho jayegi.
2. Aapka API bill bohot zyada aayega kyunki har naye message par aap purane hazaron tokens ka paisa wapas de rahe ho.

8. Memory vs Database storage

- **LangChain Memory (e.g., InMemoryChatMessageHistory):** Yeh RAM mein hoti hai. Jaise hi aapka Python server restart hoga, memory flush ho jayegi (Zero persistence).

- **Database Storage (Redis / MongoDB / PostgreSQL):** Production mein, hum memory session IDs banate hain aur history ko external databases mein save karte hain. LangChain natively Redis, Postgres, etc. ke history classes support karta hai.

Module 3: Building a Chatbot with Memory (Topic 9)

Ab hum modern LangChain approach (RunnableWithMessageHistory) use karke ek production-ready memory chatbot banayenge.

Complete Python Example (The Modern Approach)

```
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

# 1. Load keys
load_dotenv()
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# 2. Create Prompt Template with a "MessagesPlaceholder"
# The placeholder is where LangChain will inject the chat history array
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert AI software engineering mentor."),
    MessagesPlaceholder(variable_name="chat_history"), # Memory goes here
    ("human", "{input}") # New user input goes here
])

# 3. Create the standard chain (LCEL)
chain = prompt | llm

# 4. Create a session storage mechanism (Simulating a DB in RAM)
# In production, this dictionary is replaced by Redis or a SQL Database
session_store = {}

def get_session_history                str
```

```
    if session_id not in session_store:
        session_store[session_id] = InMemoryChatMessageHistory()
    return session_store[session_id]

# 5. Wrap the chain with Message History logic
memory_chain = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",       # The key for new user message
    history_messages_key="chat_history" # The key matching the placeholder
)

# --- DRY RUN & EXECUTION ---
if __name__ == "__main__":
    config = {"configurable": {"session_id": "user_session_101"}}

    # Turn 1
    print("User: Hi, my favorite sorting algorithm is Merge Sort.")
    response_1 = memory_chain.invoke(
        {"input": "Hi, my favorite sorting algorithm is Merge Sort."},
        config=config
    )
    print(f"AI: {response_1.content}\n")

    # Turn 2
    print("User: What is my favorite algorithm?")
    response_2 = memory_chain.invoke(
        {"input": "What is my favorite algorithm?"},
        config=config
    )
    print(f"AI: {response_2.content}\n")
```

Code Execution Dry Run

1. **Line 17:** MessagesPlaceholder ek special tag hai. Yeh template ko batata hai ki "Yahan par list of messages aayenge".
2. **Line 26:** session_store ek dictionary hai. Production mein API stateless hoti hai, toh front-end se hum session_id bhejte hain, aur backend database se history utha kar laata hai.
3. **Line 33:** RunnableWithMessageHistory magic karta hai. Jab invoke call hota hai, yeh pehle get_session_history ko bulata hai, us list ko uthata hai, prompt ke placeholder mein daalta hai, chain run karta hai, aur phir naye response ko automatically wapas InMemoryChatMessageHistory mein save kar deta hai.

Input and Expected Output

Input 1: "Hi, my favorite sorting algorithm is Merge Sort."
Output 1: "Hello! Merge Sort is an excellent algorithm, famous for its O(n log n) time complexity. How can I help you with it today?"
Input 2: "What is my favorite algorithm?"
Output 2: "Your favorite sorting algorithm is Merge Sort!" *(Proof that memory works!)*

Common Mistakes Beginners Make

1. **Missing Placeholder:** Prompt template mein MessagesPlaceholder add karna bhool jana. Iske bina memory chain input reject kar degi.
2. **Key Mismatch:** history_messages_key mein kuch aur likh dena (e.g., "history") par prompt template mein variable name kuch aur likhna (e.g., "chat_history"). Dono exact match hone chahiye.
3. **Global Memory:** session_id logic na lagana aur ek single buffer object ko saare users ke liye share kar dena. Isse User A ki private chat User B ko dikhne lagegi!

Review & Practice Segment

Interview Questions & Answers

Q: How does LangChain prevent context window exhaustion when using memory?
Answer: By default, standard Buffer Memory does not prevent exhaustion. To prevent it, we must use ConversationBufferWindowMemory (which trims the array to the last 'k' messages) or ConversationSummaryMemory (which calls the LLM in the background to compress the chat history into a short summary paragraph).
Q: Explain the difference between LLMChain memory and RunnableWithMessageHistory.
Answer: In the legacy LLMChain, memory was tightly coupled inside the chain object as a stateful variable. In modern LCEL, chains are stateless runnables. RunnableWithMessageHistory is a wrapper that intercepts the input, explicitly fetches history from an external storage mechanism based on a session_id, and passes it into the prompt. It enforces statelessness and easier database integration.

Small Practice Exercises

- **Task 1:** Modify the code above to create a second config with session_id: "user_session_102". Ask the AI what your favorite algorithm is in session 102. It should fail to answer, proving session isolation.
- **Task 2:** Add a StrOutputParser() into the chain pipeline prompt | llm | parser so you don't have to extract .content from the final AIMessage manually.

Memory makes our LLM feel "alive" and interactive. Ab kyunki memory history array ka size limit cross kar sakti hai, kya hum next topic mein **ConversationBufferWindowMemory** aur **SummaryMemory** ko details mein code karke dekhein taaki API costs aur context window limits ko handle kiya ja sake?